TOBB ETÜ

Ekonomi ve Teknoloji Üniversitesi

ROADRUNNER

Final Report

- Utku Ceylan
- Mustafa Tufan
- Ahmet Salih Kaya
- Batuhan Efe Alkış
- Doruk Demirci

TABLE OF CONTENTS

1. Introduction....4
1.1 Purpose of the Project....4
1.2 Problem Definition....4
1.3 Project Objectives....4
1.4 Scope of the Final Report....5
2. Final System Overview....5
2.1 System Summary....5
2.2 Main Functionalities....5
2.3 Final Project Status....6
2.4 Example Usage Scenario....6
3. Final Architecture and Design....7
3.1 Architectural Overview....7
3.2 Major Components....7
3.2.1 Backend API Layer....7
3.2.2 Route Generation Module....7
3.2.3 LLM Integration Module....8
3.2.4 POI / Places Module....8
3.2.5 User Module....8
3.2.6 Frontend Modules....8
3.3 Final Data Flow....9
3.4 Design Decisions and Trade-offs....9
3.5 Differences from Initial / High-Level Design....10
4. Implementation Status and Completed Features....12
4.1 Implemented Features....12
Feature 1 - User Authentication....12
Feature 2 - Travel Persona Management....12
Feature 3 - Route Generation....13
Feature 4 - Itinerary Modification....13
Feature 5 - Chat-based Planning....14
Feature 6 - POI Browsing and Filtering....14
Feature 7 - Saved Routes....15
Feature 8 - Map Visualization and Navigation....15
4.2 Unimplemented or Partially Implemented Features....16
4.3 Deployment / Website Status....17
5. Tools and Technologies Used....18
5.1 Backend Technologies....18
5.2 Frontend Technologies....18
5.3 AI / Data Technologies....18

5.4 Database / Storage Technologies....19
5.5 Development and Deployment Tools....19
5.6 Why These Technologies Were Chosen....19
6. Test Results and Evaluation....20
6.1 Testing Approach....20
6.2 Test Environment....20
6.3 Test Case Results....20
Overall Summary....21
Security and Authentication Tests....21
User Management Tests — Unit....22
User Management Tests — Integration....23
Route Generation Tests — Unit....24
Route Generation Tests — Integration....25
LLM Agent Tests — Unit....26
LLM Agent Tests — Integration....27
POI Module Tests — Unit....27
POI Module Tests — Integration....28
Performance Tests....29
Client-Side Tests: Unit....29
Client-Side Tests: Integration....30
User Testing....30
System Tests....31
6.4 Detailed Test Result Notes....31
6.5 Summary of Test Results....32
6.6 Bugs Found and Fixes Applied....32
6.7 Potential Enhancements....32
7. Impact of Engineering Solutions....33
7.1 Global Context....33
7.2 Economic Context....33
7.3 Environmental Context....33
7.4 Societal Context....34
8. Contemporary Issues....35
8.1 AI Reliability and Hallucination....35
8.2 Privacy and Data Protection....36
8.3 Bias and Fairness in Recommendations....36
8.4 Dependence on Third-Party Platforms....36
8.5 Ethical Transparency....37
9. Use of Library and Internet Resources....37
9.1 Background Research Sources....37
9.2 Similar Systems and Existing Solutions....38
9.3 Engineering Principles and External References....38

9.4 Lifelong Learning / Self-Learning Aspect.....39
10. Limitations and Future Work.....40
10.1 Current Limitations.....40
10.2 Future Improvements.....41
11. Conclusion.....42
12. References.....42

---

1. Introduction

1.1 Purpose of the Project

Modern travel planning is a fragmented and time-consuming process that typically yields generic itineraries. The RoadRunner project eliminates this inefficiency by deploying a centralized, AI-powered travel assistant capable of end-to-end, personalized route generation.

By leveraging natural language processing and deep learning, the system extracts nuanced preferences and constraints from a conversational "user story" to construct dynamic user profiles. It then integrates these profiles with data from third-party APIs such as Google Places to calculate optimized schedules. To maintain operational trust, the architecture incorporates Explainable AI (XAI), providing transparent reasoning for every generated recommendation. Ultimately, the system transforms a complex logistical chore into a seamless, highly tailored, and user-centric experience.

1.2 Problem Definition

While current travel applications provide rudimentary search and filtering capabilities, they fundamentally fail to interpret nuanced individual preferences. Conventional systems cannot adequately accommodate specific travel styles, pacing requirements, or niche interests such as architecture and local culinary experiences. Consequently, the burden of logistical complexity falls entirely on the user, adding a layer of difficulty to an activity that should inherently be enjoyable. The fundamental problem lies in the absence of a centralized, intelligent platform capable of translating subjective user desires into highly tailored itineraries, leaving travelers reliant on scattered data and generalized recommendations rather than personalized exploration.

1.3 Project Objectives

The main objectives of the project are:

- To collect user preferences through natural language or UI-based interaction

- To generate personalized travel itineraries
- To support route visualization and itinerary editing
- To provide explainable recommendations
- To maintain user profile and preference data for future personalization
- To ensure GDPR and KVKK compliance for user data privacy

1.4 Scope of the Final Report

This report presents the project's final state, defining the final architecture, design, and operational status of the RoadRunner system. It critically evaluates the global, economic, environmental, and societal impacts of the implemented engineering solutions. The document addresses contemporary issues relevant to AI-driven itinerary planning and explicitly details the modern tools and technologies integrated throughout the development lifecycle. Furthermore, it documents the library and internet resources utilized to establish background context, analyze similar designs, and apply fundamental engineering principles. Finally, the scope encompasses an analysis of test results derived from the established test plan; this includes specific case descriptions, pass or fail outcomes, statistical assessments of failures, bug discussions, and projected system enhancements for the future.

2. Final System Overview

2.1 System Summary

RoadRunner is a web-based travel planning system that integrates a conversational AI interface, an interactive map interface, a Point of Interest (POI) data layer, route generation logic, and user profile management. Designed primarily with an Ankara focus, the system accepts trip-related preferences from the user in natural language and produces one or more feasible itinerary alternatives.

It operates as an intermediary between the user and an ecosystem of external data providers including Google Places statically, Wikipedia, and OpenStreetMap. Beyond static generation, the platform allows manual drag-and-drop itinerary manipulation and employs Explainable AI (XAI) to ensure the logic behind destination recommendations remains transparent to the user.

2.2 Main Functionalities

The final system supports the following core functions:

- Parsing user travel preferences from natural language input ("User Stories")
- Generating multiple alternative travel routes with optimized POI sequences
- Displaying routes and POIs on an interactive Leaflet.js map
- Allowing manual itinerary edits (add, remove, reorder, replace POIs)
- Storing user profiles and multiple travel personas
- Providing AI-based (XAI) explanations for recommendations (places and route)

- Accepting user feedback (star ratings, comments) to refine future recommendations

2.3 Final Project Status

At the end of the project, the system has reached the following status:

Feature	Status
User Authentication	Completed
Travel Persona Management	Completed
Route Generation	Completed
POI Exploration and Filtering	Completed
Chat-based Planning (LLM Agent)	Completed
Itinerary Modification (Add/Remove/Reorder)	Completed
Feedback Mechanism	Completed
Local Deployment	Completed
GitHub Website Integration	Completed
Cloud Deployment (Production URL, AWS Hosting)	Not Completed (Local Docker deployment is used instead)

2.4 Example Usage Scenario

A user enters a travel request such as: "Plan me a weekend trip in Ankara with museums, quiet cafes, and historical places."

The system:

1. Extracts constraints and interests using NLP
2. Retrieves relevant POIs from the internal PostgreSQL database (seeded from Google Places API)
3. Generates route alternatives optimized by preference weights
4. Visualizes them on the interactive Leaflet map
5. Allows the user to modify the plan via chat or drag-and-drop UI
6. Explains why certain recommendations were made (XAI Justification Agent)

3. Final Architecture and Design

3.1 Architectural Overview

The final architecture is structured around three primary layers: a unified Spring Boot REST API, a Vite/React frontend, and a supplementary Flask-based Large Language Model (LLM) server. The backend, written in Java and Spring Boot, operates as a modular monolith encompassing internal packages for user management, POI retrieval, route generation, and security. It uses PostgreSQL as the primary persistence layer for storing structured POI data, user personas, saved itineraries, and chat histories.

The client-side application is built with Vite.js, React, and TypeScript. The frontend uses Redux Toolkit and React Redux for centralized state management. Communication between the client and backend occurs strictly through RESTful APIs using JSON payloads and is secured by stateless JWT authentication. Furthermore, an isolated Python/Flask proxy server processes natural language queries and executes tool-calling agents independently, preventing the main Java application threads from being blocked. Figure 1 presents a component-level diagram of

The three-layer architecture described below.

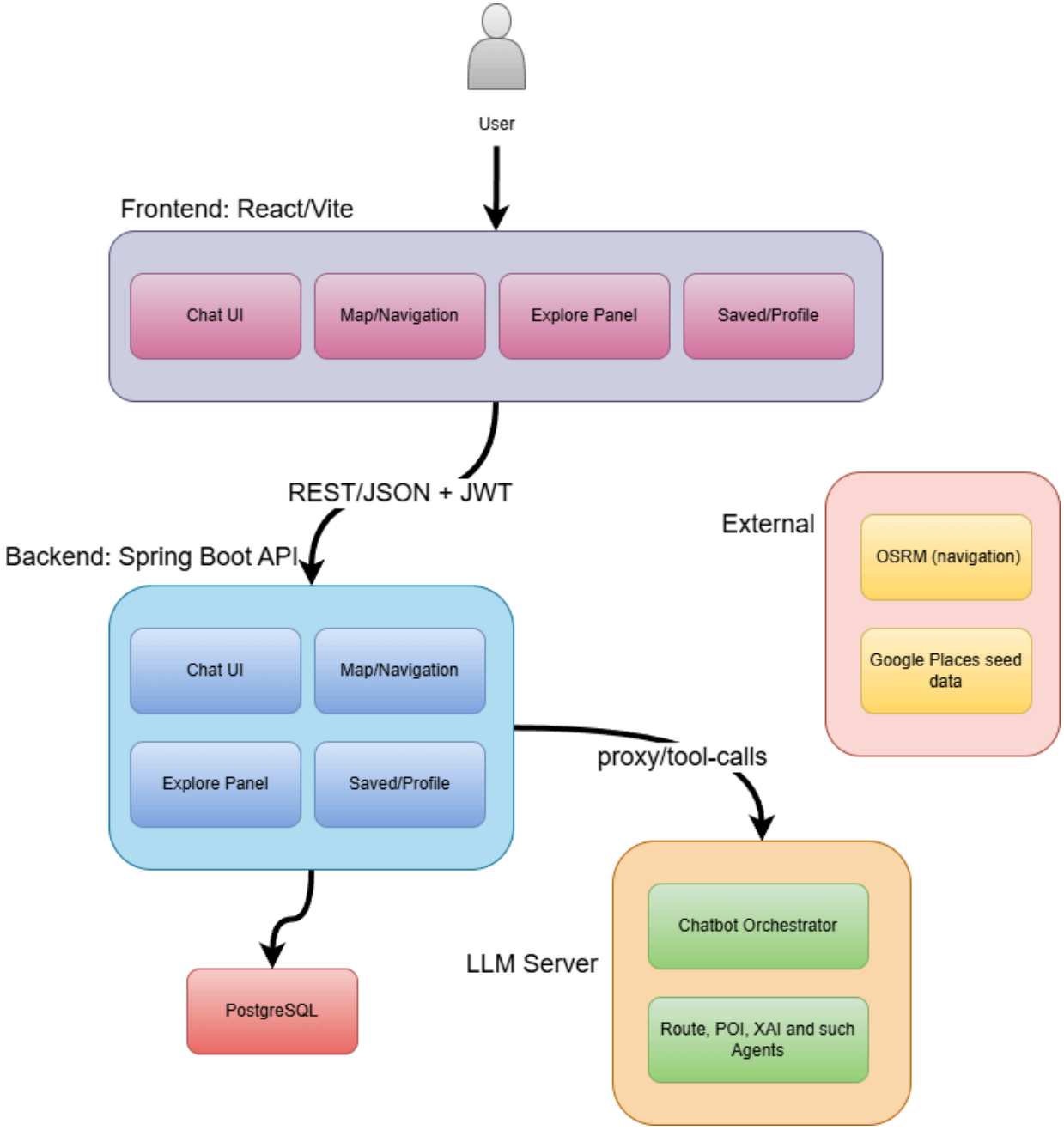

Figure 1. RoadRunner three-layer architecture.

3.2 Major Components

3.2.1 Backend API Layer

The Backend API Layer acts as the central orchestrator and entry point for the system, abstracting internal complexities from the client application. Built upon Spring Boot, it strictly validates incoming HTTP requests using Spring Validation and manages stateless communication via JSON payloads. It routes authenticated requests, secured by a JWT filter, to appropriate internal services such as the routing engine, database management layer, or the LLM proxy. Finally, it aggregates processed data and asynchronous tool-calling results into unified synchronous REST responses for the frontend UI.

3.2.2 Route Generation Module

This computational engine strictly handles routing logic without persisting results directly to the database. Route requests are formulated using a specific user preference vector alongside hard constraints, such as mandatory POI slots, specific hotel anchors, or designated meal stops. To select suitable POIs, the module filters candidates based on business status and dynamically scores them against category weights, budget alignment, and spatial proximity. It produces multiple route alternatives by enforcing category quotas and applying deterministic noise to penalize overlapping selections. Furthermore, the module supports manual itinerary modifications, processing existing routes to execute targeted reroll, insert, remove, and reorder operations while recalculating segment distances and durations.

3.2.3 LLM Integration Module

This module orchestrates all interactions with the Large Language Model. The system employs a proxy architecture where the Spring Boot backend forwards user requests and chronological conversation history to a Python/Flask LLM server. The chatbot processes natural language input utilizing a robust tool-calling framework, dynamically determining whether to formulate a direct conversational reply or to invoke specialized AI agents. By mapping natural language intent into structured actions, these dedicated agents autonomously execute tasks such as POI retrieval, route generation, user profile modifications, and providing Explainable AI (XAI) justifications including but not limited to explanation of why certain places were suggested for a certain user.
The LLM models (Qwen, Nemotron) are run locally on the team's GPU server rather than accessed via a cloud API; this architectural decision, its trade-offs, and its environmental implications are discussed in 3.5 and 7.3.

3.2.4 POI / Places Module

Serving as the system's geographical source of truth, this module normalizes and stores Ankara-specific Point of Interest data within a PostgreSQL database, which is seeded initially from Google Places datasets. It provides REST endpoints that allow the frontend and route

generation modules to query locations efficiently. The module supports advanced querying capabilities, including partial name searches, bulk ID retrieval, and precise filtering mechanisms based on POI type, minimum rating, and price level. By maintaining this structured data internally, the system guarantees data consistency for the routing algorithms and immediate spatial rendering on the UI.

3.2.5 User Module

The User module governs persistent application state and security, employing BCrypt password hashing and JWT-based stateless authentication integrated directly with Google reCAPTCHA. It securely stores customizable user profiles alongside multiple distinct travel personas, mathematically mapping subjective travel preferences into numerical vectors. Additionally, this module persists complex relational data, including chat histories, categorized saved destinations, and complete JSONs of generated routes, ensuring session restoration and KVKK-compliant data management.

3.2.6 Frontend Modules

The React-based frontend is structurally divided into highly interactive, synchronized panels managed by Redux Toolkit. The Chat UI Module provides the primary conversational interface for continuous LLM interactions. Spatially, the Map Panel utilizes Leaflet and Marker Cluster to render dynamic route polylines and geographical POIs. The Explore Panel facilitates destination discovery via extensive filters, while the Saved Panel and Destination Detail Panels allow users to organize and review specific locations. Finally, the Navigation Panel calculates and displays localized transit routes between saved waypoints by querying the OSRM public demo server.

3.3 Final Data Flow

The system's data flow operates through a synchronized, multi-layered pipeline that connects user interactions to computational routing logic:

1. The process begins when the user submits travel preferences via natural language in the Chat UI or structured inputs within the Persona Builder interface.
2. The React client translates these inputs into a structured JSON payload and transmits an HTTP POST request to the Spring Boot backend via an Axios interceptor, appending the JWT for authentication.
3. Upon receipt, the backend validates the request. The User module extracts the relevant user vector (persona weights), while the Route Generation module resolves explicit constraints (hotel anchors, mandatory POIs, or meal slots etc) into a normalized Route Constraint Spec.
4. The POI/Places module queries the PostgreSQL database for candidate locations, instantly filtering out closed businesses and locations that fail to meet minimum rating thresholds.

5. The Route Generation module scores the remaining candidates based on proximity, budget alignment, and category weights. It then applies pathfinding logic to construct multiple itinerary alternatives, penalizing overlap to ensure variety.
6. The generated routes, complete with segment durations and total distance metrics, are serialized and returned to the frontend. The UI immediately renders the routes as Editable Route Card components and updates the Leaflet map.
7. If the user executes a manual edit (drag-and-drop reordering, rerolling a specific POI, or inserting a new stop), the frontend transmits the current route alongside the requested change. The backend recalculates the segments and returns the updated itinerary without requiring a full regeneration, maintaining a fast feedback loop.

3.4 Design Decisions and Trade-offs

- **Efficiency vs. Accuracy:** Route generation uses a nearest-neighbor heuristic with category-weighted scoring rather than solving the Traveling Salesman Problem exactly. Route segments use Haversine straight-line distance with mode-specific speed estimates (walking, driving) for time calculation. OSRM is not called from the backend during route generation; it is invoked exclusively by the frontend NavigationPanel for user-facing turn-by-turn geometry, with a Haversine fallback when OSRM returns empty geometry (common in pedestrian mode for sparse map areas). Routes are computed statelessly: the Route, RoutePoint, and RouteSegment objects are not JPA entities and are never persisted to the database during generation. The client carries the full route snapshot and sends it back for each edit operation (reroll, insert, remove, reorder). This avoids write contention and keeps generation fast, but it means the server cannot validate whether a route was tampered with client-side. The POI database is pre-seeded from CSV files at application startup rather than queried live from Google Places at runtime trading data freshness for sub-second query latency and zero API cost during normal operation.
- **Usability vs. Functionality:** The system provides two parallel interaction paths: a conversational LLM chat and a traditional button-driven UI. The chat interface hides the full complexity of the routing algorithm (weight vectors, constraint specs, category quotas) behind natural language, relying on the LLM's tool-calling mechanism to translate intent into structured API calls. However, because LLM output is non-deterministic and tool dispatch is not guaranteed to be correct on every turn, the manual UI (persona builder, constraint selectors, drag-and-drop route editor) remains the primary and more reliable path for itinerary creation. This means the team opted to forgo the single chat window vision from the HLD in favor of a dual-interface approach where the user always has deterministic fallback controls. Similarly, the TravelProfileBuilder translates human-readable preference questions into a numerical `user vector` on the frontend side, so the backend routing algorithm never needs to parse qualitative input at the cost of requiring users to fill out a multi-step form rather than expressing preferences purely through conversation.
- **Security vs. Usability:** JWT-based authentication is enforced on all endpoints except registration, login, POI browsing, and route generation. Tokens expire, forcing

re-authentication on shared devices. Passwords are hashed with BCrypt (not Argon2 or scrypt as originally considered in the HLD BCrypt was chosen for its Spring Security native support). Google reCAPTCHA v3 protects registration and login in production but is disabled in the test profile via app.recaptcha.enabled=false. The application.properties file containing database credentials, JWT secrets, and API keys is excluded from version control entirely, requiring manual setup per environment.

- **Reliability vs. Compatibility:** The frontend targets modern Chromium/Firefox/Safari browsers only. Leaflet's marker clustering, polyline rendering, and OSRM integration depend on modern JavaScript APIs, so legacy browser support was explicitly dropped to reduce client-side failure surface. On the backend side, runtime POI queries hit only the internal PostgreSQL database external API outages (Google Places etc.) do not affect plan generation since data ingestion is a separate offline process. For testing, H2 in-memory database substitutes for PostgreSQL.

3.5 Differences from Initial / High-Level Design

- **Modular monolith architecture instead of microservices:** The HLD proposed a microservices-based architecture with independently deployable services for User, Places, Route Generation, and LLM. In practice, the backend is a single Spring Boot application with internal package-level separation (user, place, route, llm). The only separate process is the Flask-based LLM server, which Spring Boot proxies via RestTemplate. The microservices vision was simplified because the team size (5 people) and timeline did not justify the operational overhead of inter-service communication, distributed tracing, and independent deployment pipelines for a project of this scale.
- **Backend OSRM integration:** The HLD described OSRM being called from the Route Generation module. In the final system, OSRM is invoked only from the frontend NavigationPanel; backend route segments use Haversine estimates.

Removed or Descoped Features

- **Booking integration** (flights, hotels, car rentals via Amadeus/Sabre/Skyscanner as described in the proposal's Topic 2): Entirely removed because of API limit and costs. The system focuses solely on itinerary planning with existing POI data, not on booking transactions.
- **Redis caching layer**: The HLD specified Redis for session state and frequently accessed queries to meet sub-500ms response targets. Redis was never implemented; PostgreSQL handles all queries directly, and performance remained acceptable for the target of 10 concurrent users.
- **RAG (Retrieval-Augmented Generation)**: Listed in the HLD glossary and implied for enhancing LLM responses with external data. Not implemented. The LLM agent instead calls backend REST endpoints as tools (POI search, route generation, persona retrieval, Wikipedia summary) rather than performing vector-similarity retrieval over a document store.

- **Real-time data collection**: The proposal mentioned possible real-time data ingestion depending on hardware availability. The final system uses static CSV files seeded at startup, with no live data pipeline.

Simplified Assumptions

- **Geographic scope**: The HLD described a system for "any selected region." The final system is Ankara-only, with seven pre-defined POI categories seeded from Google Places data. Expansion to other cities would require re-running the data collection scripts and adding new CSV files.
- **Password hashing**: HLD specified Argon2 or scrypt. Implementation uses BCrypt due to Spring Security's out-of-the-box BCryptPasswordEncoder.
- **LLM provider**: HLD listed "Google Gemini / OpenAI API." The final system uses local Qwen/Nemotron models for tool-calling on the Flask side.
- **Route persistence**: The HLD implied routes would be serialized and stored. In the final system, routes are stateless computation results. Only when a user explicitly saves a route does the SavedRoute entity persist the full JSON and the original generation request.

Added Components Not in Original Design

- **WeatherAgent**: A weather information tool was added to the LLM agent pipeline, currently returns a mock variable.
- **Saved Routes with Snapshot Hydration**: The ability to save a complete route snapshot (including the original GenerateRoutesRequest) and later reconstruct the exact same route state in the frontend was an implementation-driven addition to support the "revisit saved plans" use case.
- **Category-based POI labeling system**: RouteLabel and DefaultPlaceRouteLabelService map Google Place types to semantic route categories (park, nightlife, restaurant, landmark etc.). This labeling layer was not described in the HLD but became essential for the weighted scoring algorithm.
- **Separated category tables**: In addition to the main places table, SeparatedPlaceDataLoader seeds category-specific tables from the same CSV sources, an optimization not in the original design.
- **Allure test reporting**: Added for generating visual test reports, not mentioned in any design document.

4. Implementation Status and Completed Features

4.1 Implemented Features

Feature 1 - User Authentication

- Description: Full registration and login flow with JWT-based stateless session management. Passwords are hashed with BCrypt. Google reCAPTCHA v3 is enforced on registration and login in production to prevent bot abuse.
- Current status: Fully implemented and tested.
- Notes on implementation: AuthService handles reCAPTCHA verification, email uniqueness checks, BCrypt hashing, and JWT generation in a single transactional flow. The JWT token is stored in the browser's localStorage; an Axios interceptor attaches it to every subsequent request. JwtAuthenticationFilter validates the token on each request and populates the Spring Security context. reCAPTCHA is disabled in the test profile via app.recaptcha.enabled=false. Token expiration forces re-authentication. Endpoints for registration, login, POI browsing, and route generation are publicly accessible; all others require a valid JWT. Tests cover duplicate email rejection, password hash verification, JWT generation/validation, and security context population (AuthServiceTest, AuthControllerIntegrationTest, JwtTokenProviderTest, JwtAuthenticationFilterTest).

Feature 2 - Travel Persona Management

- Description: Users can create, edit, delete, and switch between multiple travel personas. Each persona captures qualitative preferences (tempo, social preference, nature/history interest, food importance, alcohol preference, transport style, budget level, trip length, crowd preference) and a computed numerical user vector used directly by the routing algorithm.
- Current status: Fully implemented and tested.
- Notes on implementation: The TravelProfileBuilder React component collects human-readable answers. The frontend utility buildRouteUserVector in travelProfile.ts converts these answers into a key-value weight map (nature preference → weight_dogalAlanlar, food importance → weight_restoranToleransi). Both the qualitative answers and the derived userVector are persisted in the TravelPersona entity via POST /api/users/me/personas/new. Users can designate a default persona; authSlice manages the active persona and preserves default ordering across sessions. The backend includes guard logic to clean up legacy or malformed persona records. Tests are covered in UserServiceTest and UserControllerIntegrationTest.

Feature 3 - Route Generation

- Description: The core planning engine. Given a user's preference vector and a set of constraints (start/end anchors, hotel loop, mandatory POI slots, meal stops, desired visit

count), the system generates k alternative itineraries with ordered stops, estimated durations, distances, and a feasibility flag.
- Current status: Fully implemented and tested.
- Notes on implementation: RouteGenerationService parses the userVector, builds a candidate POI pool from PostgreSQL, computes category quotas from weight distribution, scores candidates via RouteScoringService (rating quality, review confidence, budget compatibility, anchor proximity, category weight, controlled randomness), and sequences stops using nearest-neighbor heuristics. Meal stops are redistributed if consecutive. When generating multiple alternatives, the algorithm penalizes POI and hotel overlap with previously generated routes and applies deterministic noise to encourage distinct yet consistent alternative itineraries.Supported start/end anchor types are NONE, HOTEL, PLACE, and TYPE. Routes are stateless; Route, RoutePoint, and RouteSegment are plain computation objects, not JPA entities. Segments use Haversine distance with mode-specific speed estimates (walking, driving). Tests span RouteGenerationServiceTest (hotel loops, category quotas, alternative diversity, center-start, constraints, meal stops, mutations), RouteScoringServiceTest (candidate filtering, hotel bias, quality confidence, budget compatibility, sparsity penalty, visit duration), RouteControllerIntegrationTest (endpoint validation, boundary checks), and RouteGenerationPerformanceTest (single and triple route generation time limits).

Feature 4 - Itinerary Modification

- Description: After route generation, users can edit any itinerary through four operations: reroll (replace one stop with a same-category alternative), insert (add a user-selected POI at a specific position), remove (delete a stop), and reorder (change the sequence of stops). All operations preserve protected points (hotel anchors, meal stops, start/end points).
- Current status: Fully implemented and tested.
- Notes on implementation: Each edit operation is a stateless round-trip. The frontend sends the full current route (via InsertWithStateRequest, RemoveWithStateRequest, RerollWithStateRequest, or ReorderWithStateRequest), the backend reconstructs the in-memory Route object, applies the mutation, recomputes only the affected segments and aggregate metrics (total distance, total duration, feasibility), and returns the updated route. The frontend's EditableRouteCard component supports drag-and-drop reordering and exposes reroll/delete/add buttons per stop. PoiSelectorDialog provides a filterable POI picker for manual insertion. Protected points (marked via fixedAnchor, protectedPoint, protectionReason on RoutePoint) cannot be rerolled or removed. Tests are included in RouteGenerationServiceTest and RouteControllerIntegrationTest (reroll, insert, remove, reorder endpoints).

Feature 5 - Chat-based Planning

- Description: A conversational interface powered by an LLM agent pipeline. Users type natural language requests; the system dispatches to specialized tool-calling agents that

can search POIs, generate routes, read user personas, provide weather information, and explain recommendations.

- Current status: Implemented with functional tool-calling pipeline. LLM responses are non-deterministic and agent dispatch is not guaranteed to be correct on every turn, so the manual UI remains the more reliable path.
- Notes on implementation: The architecture has three layers. The React ChatPanel sends messages to Spring Boot's /api/llm/chat endpoint. LLMService proxies the request to a Flask server running at llm.server.url. The Flask server hosts the chatbot orchestrator, which consults the LLM (Qwen/Nemotron), detects tool-call requests in the response, executes the matching agent, appends the tool result to the conversation, and sends it back to the LLM for a final response pass. Registered agents include: POI search by category, POI detail lookup, route generation, persona retrieval, weather lookup, XAI justification, user feedback, and calculator. Tool results can include structured LocationCard data rendered inline in the chat. All messages (including tool call metadata) are persisted via ChatService in the Chat and Message entities with chronological ordering. Chat ownership isolation ensures users only see their own conversations. Tests cover ChatServiceTest, ChatControllerIntegrationTest, and ChatRepositoryIntegrationTest (CRUD, message ordering, ownership isolation, updatedAt behavior).

Feature 6 - POI Browsing and Filtering

- Description: The Explore panel allows users to discover Ankara POIs through search and multi-criteria filtering. POI data is seeded from Google Places API exports at application startup and served from the internal PostgreSQL database.
- Current status: Fully implemented and tested.
- Notes on implementation: Seven category CSV files (bars_nightclubs, cafes_desserts, historic_places, hotels, landmarks, parks, restaurants) under backend/src/main/resources/ankara_places/ are loaded by PlaceDataLoader into a single places table on startup. SeparatedPlaceDataLoader additionally seeds category-specific tables. The PlaceController exposes: paginated listing with type, minRating, and priceLevel filters (GET /api/places); single POI lookup (GET /api/places/{id}); partial name search with Turkish character normalization (GET /api/places/search?name=...); bulk ID retrieval (POST /api/places/bulk); and category-based top-POI listing (GET /api/places/by-category?category=...&size=...). The frontend ExplorePanel combines these filters with client-side controls for review count, business status, and Google Place type. Results are displayed as a list alongside a Leaflet map with marker clustering. Clicking a marker opens DestinationDetailPanel with full POI metadata and a save button that syncs to the user's TravelPlan via savedSlice. Tests in PlaceControllerIntegrationTest cover pagination, filtering, name search, bulk retrieval, and response payload structure; PlaceServiceTest covers service-layer logic and mapper behavior.

Feature 7 - Saved Routes

- Description: Users can save generated and edited routes for later retrieval. Each saved route stores a complete route (all stops, segments, metrics) and the original generation request, allowing exact reconstruction of the route state.
- Current status: Fully implemented and tested.
- Notes on implementation: When a user saves a route, a SavedRouteWritePayload is sent containing the route JSON, the GenerateRoutesRequest JSON, a title, ordered POI IDs, and summary metrics (total stops, distance, duration). The SavedRoute entity persists all of this. The list view shows summary fields; opening a detail view deserializes the snapshot and hydrates routeSlice in the frontend, restoring the exact route state including all edit history. Users can rename and delete saved routes. Cross-user access is blocked, ownership is enforced at the service layer. Tests in SavedRouteControllerIntegrationTest cover snapshot persistence, summary retrieval, rename, delete, and cross-user security isolation.

Feature 8 - Map Visualization and Navigation

- Description: An interactive Leaflet-based map displaying POIs as clustered markers, route polylines for generated itineraries, and turn-by-turn navigation between selected stops.
- Current status: Fully implemented. Navigation relies on OSRM public API with a straight-line fallback.
- Notes on implementation: MapPanel renders Leaflet tiles with react-leaflet and marker clustering. POI markers show category-colored icons; clicking opens a popup with metadata and a save button. When a route is active, EditableRouteCard stop data is drawn as a polyline overlay. NavigationPanel sends selected waypoints to the OSRM public demo server for road-following geometry; if OSRM fails (network error, sparse map data, pedestrian mode gaps), routingService.ts falls back to straight-line segments between coordinates. ResizableDivider allows the user to drag the boundary between the chat and map panels. The map supports fullscreen toggle via chatSlice state.

4.2 Unimplemented or Partially Implemented Features

The following planned features were not fully completed within the project timeline. The table below summarizes the final status of each feature along with the reason for its incompletion.

Feature	Final Status	Reason

Real-time live external API refresh	Partial	The system uses a static internal PostgreSQL database seeded from Google Places API at startup. Live API calls during runtime were not implemented due to time, cost and integration constraints. POI data reflects a point-in-time snapshot.
Accessibility labeling (age-range / wheelchair tagging)	Partial	The `Place` entity schema does not include age-range or disability access fields. Google Places API data used for seeding does not consistently provide these attributes, creating a dataset limitation.
Weather-aware planning	Not completed	The WeatherAgent was defined in the LLM agent architecture but was not integrated with a live weather data provider. The Flask LLM server remained at prototype level and weather-based route scheduling was excluded from the final scope.
Dedicated cloud deployment	Partial	The system is containerized with Docker and runs correctly in a local environment. A publicly accessible cloud deployment was not finalized due to infrastructure and configuration time constraints. The project web page is hosted via GitHub Pages only.
Route segment geometry (OSRM turn-by-turn in backend)	Partial	Route segments currently store only straight-line distance and estimated duration estimates. OSRM integration is used exclusively in the frontend `NavigationPanel` for user-facing navigation. Backend `RouteSegment` objects do not carry path geometry, as full OSRM server-side integration was not completed within the project timeline.

Trip price estimation	Not completed	While price level data (INEXPENSIVE / MODERATE / EXPENSIVE) is stored per POI in the database, a cumulative trip cost estimation feature was not implemented in either the backend route generation module or the frontend UI.

4.3 Deployment / Website Status

The system is containerized with Docker and docker-compose and runs reproducibly in a local environment. A publicly accessible cloud deployment with a dedicated production URL was not finalized within the BİL496 timeline; infrastructure provisioning and domain configuration were deprioritized to focus on feature completion and testing.

The project website is hosted as a static site on GitHub Pages and presents the project overview, team members, and links to the source repository. It is the only publicly accessible surface of the project; the application itself requires local build and run via the provided Dockerfile and docker-compose.yml . The website only serves as an information providing medium about the project and all of the reports in addition to information on team members.

Public accessibility is therefore limited to documentation. The full application (backend, frontend, and LLM server) is runnable only in a local or self-hosted environment.

5. Tools and Technologies Used

5.1 Backend Technologies

- Java & Spring Boot: The primary backend framework. Spring Web provides the REST controller layer, Spring Data JPA handles database access with repository abstractions and Spring Security manages JWT-based authentication and endpoint authorization.
- Lombok: Reduces boilerplate across entity and DTO classes (getters, setters, constructors, builders).
- Python 3 & Flask: The LLM agent server is a separate Flask process exposing a /chatbot endpoint. Spring Boot proxies chat requests to this server via RestTemplate. The Flask side hosts the tool-calling orchestrator, agent registry, and LLM client logic.
- REST & JSON: All inter-module communication uses JSON over HTTP. The frontend-backend contract is defined through request/response DTOs (GenerateRoutesRequest, RouteResponse, PlaceResponse, etc.). The Spring Boot-to-Flask proxy also uses JSON payloads.

5.2 Frontend Technologies

- React & TypeScript: The frontend is a single-page application built with React and TypeScript for type-safe component development and service layer contracts.
- Vite: Build tool and development server. Provides fast HMR (Hot Module Replacement) during development and optimized production bundles.
- Redux Toolkit & React Redux: Used for global state management instead of passing props between classes or keeping state inside them.
- React Router: Client-side routing between pages
- Leaflet & React-Leaflet with Marker Cluster: Interactive map rendering. POI markers are displayed with category-colored icons inside marker clusters for performance at scale. Route polylines are drawn as overlays.
- Material UI / Joy UI: Component library for consistent styling, theming (light/dark mode), and layout primitives. Joy UI provides the card, button, input, modal, and dialog components used across all pages.

5.3 AI / Data Technologies

- Tool-Calling / Agent Architecture: The Flask chatbot orchestrator implements a tool-calling loop: the LLM receives user input plus conversation history, optionally emits a structured tool-call request, the orchestrator executes the matching agent function, appends the result to the conversation, and sends it back to the LLM for a final natural language response.
- Qwen / Nemotron Models: Open-source LLMs are used for tool-calling on the Flask side.
- NLP-based Parsing: The LLM performs intent extraction and entity recognition from natural language. The system does not use a separate NLP library; all parsing relies on the LLM's native capabilities via prompt engineering and tool-call output parsing.
- Scoring and Recommendation Logic: RouteScoringService implements a multi-factor scoring model: rating quality, review count confidence, category weight from the user vector, budget compatibility (price level vs. user budget preference), geographic proximity to the route anchor (with Kızılay center bias for hotel selection), sparsity penalty, and controlled randomness.

5.4 Database / Storage Technologies

- PostgreSQL: Holds all data required for the system to work. Accessed through Spring Data JPA repositories with custom query methods for pagination, filtering, and bulk retrieval.
- H2 In-Memory Database: Used exclusively in the test profile as a PostgreSQL substitute. All integration tests (*IntegrationTest classes) run against H2
- Persistence Strategy: User data, personas, chats, messages, and saved routes are standard JPA entities with CrudRepository / JpaRepository access. Routes generated by the algorithm are not persisted; they exist as in-memory objects returned to the client.

5.5 Development and Deployment Tools

- Git & GitHub: Used for version control and project hosting.
- Postman / testing utilities
- Maven: Java build and dependency management for the Spring Boot backend. Surefire plugin runs JUnit tests. Allure Maven plugin generates visual test reports.
- npm & Vite: JavaScript dependency management and frontend build tooling.
- JUnit 5 & Mockito: Java testing framework. Unit tests use Mockito for mocking service dependencies.
- Allure Test Reporting: Generates visual HTML test reports from JUnit results. Configuration lives in the Maven POM.

5.6 Why These Technologies Were Chosen

- Spring Boot: Selected for its mature ecosystem around REST API development, built-in security framework (Spring Security with native BCrypt and JWT filter chain support), and JPA/Hibernate integration for database access. The main challenge was the verbosity of the DTO layer, where the route module alone has over a dozen request/response classes, and the boilerplate required for integration tests.
- React, Vite and TypeScript: Chosen for component-based UI composition and type safety across a complex frontend with Redux slices, multiple service clients, and shared type definitions between API contracts and UI components. Vite was picked over Create React App for significantly faster build and Hot Module Reload times. The challenge was managing state complexity: the routeSlice alone handles generation requests, multiple route results, edit operations, and saved route hydration, which led to a large and tightly coupled slice.
- Leaflet & React-Leaflet: Chosen over Google Maps or Mapbox for being fully open-source with no API key cost for tile rendering. OpenStreetMap tiles are free and sufficient for Ankara coverage. Marker clustering (react-leaflet-markercluster) handles the 1000+ POI markers without performance degradation. The challenge was Leaflet's imperative API clashing with React's declarative model, custom hooks and refs were needed for programmatic map control (zoom-to-fit, polyline updates).
- PostgreSQL: Selected for reliability, SQL query expressiveness (pagination, multi-column filtering, partial text search), and JPA compatibility. JSON fields are handled via JPA AttributeConverter classes instead.
- Flask (Python): The LLM agent server uses Flask because Python has the strongest ecosystem for LLM client libraries and prototyping AI pipelines. The team experimented with multiple models (Qwen, Nemotron) and needed rapid iteration. The challenge was the operational split: running a separate Python process alongside the Java backend added deployment complexity and required a proxy layer in Spring Boot.

6. Test Results and Evaluation

6.1 Testing Approach

The testing strategy for RoadRunner follows a layered methodology spanning unit tests, integration tests, system (end-to-end) tests, performance tests, and user acceptance tests. Unit tests validate individual components in isolation. Integration tests verify correct interactions across service boundaries including the Spring Boot backend, Python LLM agent server, React frontend, and OSRM public routing service. Google Places API is not called at runtime; its role is limited to offline seeding of the POI database, so runtime integration tests do not cross this boundary. System tests validate the complete itinerary planning lifecycle from natural language input to final map rendering.

6.2 Test Environment

Hardware: Application & Staging Server with 8 vCPU, 32 GB RAM, 500 GB SSD, NVIDIA Titan GPU (24 GB VRAM) hosting the Spring Boot backend, the Flask LLM agent server, the PostgreSQL staging database, and the React frontend static bundle. The GPU is dedicated to running the Large Language Model locally for low-latency inference.

Operating Systems: Windows 11 and Ubuntu 22.04. Testing frameworks used include JUnit 5, Mockito, Spring Boot Test (for integration tests with H2), Maven Surefire plugin, Allure for report generation, and Postman for manual API verification.

6.3 Test Case Results

The following tables summarize the results of key test cases organized by module. Most of the test cases defined in the Test Plan Report were executed during the final testing phase.

Overall Summary

Module	Total	Pass	Fail	Partial	Pass Rate
Security & Authentication	11	11	0	0	**100.0%**
User Management (Unit)	22	22	0	0	**100.0%**
User Management (Integration)	44	44	0	0	**100.0%**
Route Generation (Unit)	18	18	0	0	**100.0%**
Route Generation (Integration)	19	18	1	0	**94.7%**
LLM Agent (Unit)	17	17	0	0	**100.0%**
LLM Agent (Integration)	12	11	1	0	**91.7%**
POI (Unit)	20	20	0	0	**100.0%**
POI (Integration)	20	20	0	0	**100.0%**

Performance	10	9	0	1	**90.0%**
Client-Side (Unit)	2	2	0	0	**100.0%**
Client-Side (Integration)	12	11	1	0	**91.7%**
User Testing	12	12	0	0	**100.0%**
System Tests	6	6	0	0	**100.0%**
TOTAL	**225**	**221**	**3**	**1**	**98.2%**

Security and Authentication Tests

Test ID	Description	Expected	Result
TC-SE-001	Generate token returns non-null JWT	Non-null JWT	**Pass**
TC-SE-002	Generated token contains correct userId	userId match	**Pass**
TC-SE-003	getUserIdFromToken returns correct userId	Returns userId	**Pass**
TC-SE-004	Validate token with valid token returns true	true	**Pass**
TC-SE-005	Validate token with malformed token returns false	false	**Pass**
TC-SE-006	Validate token with expired token returns false	false	**Pass**
TC-SE-007	Valid Bearer token sets authentication context	Auth set	**Pass**
TC-SE-008	Missing Authorization header → no auth	No auth	**Pass**
TC-SE-009	Non-Bearer header → no auth	No auth	**Pass**
TC-SE-010	Invalid JWT → no auth	No auth	**Pass**
TC-SE-011	Token validation exception → context cleared	Context cleared	**Pass**

User Management Tests — Unit

Test ID	Description	Expected	Result
TC-US-001	Successful user registration	AuthResponse + JWT	**Pass**
TC-US-002	Duplicate email registration → 409	409 Conflict	**Pass**

TC-US-003	Password stored as bcrypt hash	Hash ≠ plaintext	**Pass**
TC-US-004	Login with valid credentials	AuthResponse + JWT	**Pass**
TC-US-005	Login with incorrect password → 401	401 Unauthorized	**Pass**
TC-US-006	Login with nonexistent email → 401	401 Unauthorized	**Pass**
TC-US-007	Get current user, user exists	UserResponse	**Pass**
TC-US-008	Get current user, not found → 404	404 Not Found	**Pass**
TC-US-009	Update user profile name	Updated name	**Pass**
TC-US-010	Update profile, email taken → 409	409 Conflict	**Pass**
TC-US-011	Update profile, same email as current	No conflict	**Pass**
TC-US-012	Change password, correct old password	Password updated	**Pass**
TC-US-013	Change password, wrong old password → 401	401 Unauthorized	**Pass**
TC-US-014	Create TravelPlan	Plan created	**Pass**
TC-US-015	Get TravelPlan by id, valid ownership	Plan returned	**Pass**
TC-US-016	Get TravelPlan, different user → 403	403 Forbidden	**Pass**
TC-US-017	Get TravelPlan, not found → 404	404 Not Found	**Pass**
TC-US-018	Delete TravelPlan, valid ownership	Plan deleted	**Pass**
TC-US-019	Delete TravelPlan, different user → 403	403 Forbidden	**Pass**
TC-US-020	Get all chats, multiple, ordered desc	Ordered list	**Pass**
TC-US-021	Get all chats, no chats → empty	Empty list	**Pass**
TC-US-022	Get chat by id with messages	Chat + messages	**Pass**

User Management Tests — Integration

Test ID	Description	Expected	Result
TC-USI-001	Register with valid data → 201	HTTP 201	**Pass**
TC-USI-002	Register duplicate email → 409	HTTP 409	**Pass**
TC-USI-003	Register blank name → 400	HTTP 400	**Pass**
TC-USI-004	Register invalid email → 400	HTTP 400	**Pass**
TC-USI-005	Register short password → 400	HTTP 400	**Pass**
TC-USI-006	Register empty body → 400	HTTP 400	**Pass**

TC-USI-007	Login valid credentials → 200	HTTP 200 + JWT	**Pass**
TC-USI-008	Login wrong password → 401	HTTP 401	**Pass**
TC-USI-009	Login nonexistent email → 401	HTTP 401	**Pass**
TC-USI-010	Login blank password → 400	HTTP 400	**Pass**
TC-USI-011	Get current user with valid token → 200	HTTP 200	**Pass**
TC-USI-012	Get current user without token → 401	HTTP 401	**Pass**
TC-USI-013	Update profile valid → 200	HTTP 200	**Pass**
TC-USI-014	Update profile not authenticated → 401	HTTP 401	**Pass**
TC-USI-015	Change password valid → 204	HTTP 204	**Pass**
TC-USI-016	Change password wrong old → 401	HTTP 401	**Pass**
TC-USI-017	Change password not authenticated → 401	HTTP 401	**Pass**
TC-USI-018	Create persona valid → 201	HTTP 201	**Pass**
TC-USI-019	Create persona not authenticated → 401	HTTP 401	**Pass**
TC-USI-020	Get all personas → list	HTTP 200	**Pass**
TC-USI-021	Update persona valid → 200	HTTP 200	**Pass**
TC-USI-022	Update persona not found → 404	HTTP 404	**Pass**
TC-USI-023	Delete persona valid → 204	HTTP 204	**Pass**
TC-USI-024	Delete persona not found → 404	HTTP 404	**Pass**
TC-USI-025	Create plan valid → 201	HTTP 201	**Pass**
TC-USI-026	Get plan valid ownership → 200	HTTP 200	**Pass**
TC-USI-027	Get plan not found → 404	HTTP 404	**Pass**
TC-USI-028	Delete plan valid → 204	HTTP 204	**Pass**
TC-USI-029	Get all chats ordered desc	HTTP 200 ordered	**Pass**
TC-USI-030	Get chats not authenticated → 401	HTTP 401	**Pass**
TC-USI-031	Chat isolation between users	Only own chats	**Pass**
TC-USI-032	Get chat with messages	HTTP 200 + msgs	**Pass**
TC-USI-033	Get chat not found → 404	HTTP 404	**Pass**
TC-USI-034	Delete chat valid → 204	HTTP 204	**Pass**
TC-USI-035	Update chat title valid → 200	HTTP 200	**Pass**
TC-USI-036	Cascade delete plans with user	Plans removed	**Pass**

TC-USI-037	Create TravelPersona → 201	HTTP 201	**Pass**
TC-USI-038	Create persona user not found → 404	HTTP 404	**Pass**
TC-USI-039	Update persona valid ownership → 200	HTTP 200	**Pass**
TC-USI-040	Update persona not found → 404	HTTP 404	**Pass**
TC-USI-041	Delete persona valid ownership → 204	HTTP 204	**Pass**
TC-USI-042	Delete persona not found → 404	HTTP 404	**Pass**
TC-USI-043	Get all personas multiple exist	HTTP 200 list	**Pass**
TC-USI-044	Get all personas none exist → empty	HTTP 200 empty	**Pass**

Route Generation Tests — Unit

Test ID	Description	Expected	Result
TC-RG-001	Valid userVector parsed correctly	validate()=true	**Pass**
TC-RG-002	Invalid constraints prevent generation	validate()=false	**Pass**
TC-RG-003	Route within limits marked feasible	feasible=true	**Pass**
TC-RG-004	Route exceeding limits marked infeasible	feasible=false	**Pass**
TC-RG-005	Missing mandatoryTypes → rejected	feasible=false	**Pass**
TC-RG-006	Route totals calculated from segments	Correct totals	**Pass**
TC-RG-007	Segments count = points count - 1	segments=3	**Pass**
TC-RG-008	Reroll updates only selected POI	Only idx changed	**Pass**
TC-RG-009	Reroll locks all unchanged POIs	4 locked	**Pass**
TC-RG-010	Manual insertion shifts points right	Correct shift	**Pass**
TC-RG-011	Removal compacts route	Compacted	**Pass**
TC-RG-012	Reorder updates point order	New order	**Pass**
TC-RG-013	Reorder recomputes all segments	Recomputed	**Pass**
TC-RG-014	assignPOI stores correct Place	POI assigned	**Pass**
TC-RG-015	hasGeometry returns true with geometry	true	**Pass**
TC-RG-016	hasGeometry returns false without geometry	false	**Pass**
TC-RG-017	Empty route totals are zero	0, 0	**Pass**
TC-RG-018	Single-point route totals are zero	0, 0	**Pass**

Route Generation Tests — Integration

Test ID	Description	Expected	Result
TC-RGI-001	Generate k=3 alternative routes	3 Route objects	**Pass**
TC-RGI-002	Reroll changes only target point	Only idx 2 changed	**Pass**
TC-RGI-003	Manual POI insertion at correct index	POI at idx 1	**Pass**
TC-RGI-004	POI removal compresses route	Compacted route	**Pass**
TC-RGI-005	POI reordering updates route and segments	Reordered	**Pass**
TC-RGI-006	Invalid request payload → 400	HTTP 400	**Pass**
TC-RGI-007	Unsatisfiable mandatoryTypes → 422	HTTP 422	**Pass**
TC-RGI-008	Reroll out of bounds → 400	HTTP 400	**Pass**
TC-RGI-009	Insert unknown POI → 404	HTTP 404	**Pass**
TC-RGI-010	Remove invalid route → 404	HTTP 404	**Pass**
TC-RGI-011	Invalid reorder permutation → 400	HTTP 400	**Pass**
TC-RGI-012	Totals updated after reroll	Updated totals	**Pass**
TC-RGI-013	Totals updated after insertion	Updated totals	**Pass**
TC-RGI-014	Totals updated after removal	Updated totals	**Pass**
TC-RGI-015	Route objects structurally valid	Valid fields	**Pass**
TC-RGI-016	OSRM empty geometry → Haversine fallback	Fallback used	**Fail**
TC-RGI-017	POI type distribution follows weights	Correct dist.	**Pass**
TC-RGI-018	generateRoutes(k) returns exactly k	List size = k	**Pass**
TC-RGI-019	Route contains exactly maxStops points	points = 4	**Pass**

LLM Agent Tests — Unit

Test ID	Description	Expected	Result
TC-LLM-U-001	Direct text response for factual query	text response	**Pass**
TC-LLM-U-002	RouteSearchAgent selected for route intent	route_search	**Pass**

TC-LLM-U-003	POISuggestAgent for POI replacement	poi_suggest	**Pass**
TC-LLM-U-004	WeatherAgent for weather query	weather_agent	**Pass**
TC-LLM-U-005	FeedbackAgent for star rating	feedback_agent	**Pass**
TC-LLM-U-006	XAIAgent for recommendation explanation	xai_agent	**Pass**
TC-LLM-U-007	ItineraryModAgent for add-POI request	itinerary_mod	**Pass**
TC-LLM-U-008	UserProfileAgent for preference update	profile_agent	**Pass**
TC-LLM-U-009	Empty history handled without exception	HTTP 200	**Pass**
TC-LLM-U-010	Prompt injection rejected	No leak	**Pass**
TC-LLM-U-011	XAI produces context-aware explanation	Interest linked	**Pass**
TC-LLM-U-012	5-star → positive weight delta	delta > 0	**Pass**
TC-LLM-U-013	1-star → negative weight delta	delta < 0	**Pass**
TC-LLM-U-014	WeatherAgent correct API params	Params valid	**Pass**
TC-LLM-U-015	Chat title auto-generated	Title ≤ 60 chars	**Pass**
TC-LLM-U-016	CalculatorAgent evaluates valid mathematical expression	Correct numeric result	**Pass**
TC-LLM-U-017	CalculatorAgent handles invalid math (e.g., division by zero)	Calculation Error"	**Pass**

LLM Agent Tests — Integration

Test ID	Description	Expected	Result
TC-LLM-I-001	Full pipeline direct text for factual query	HTTP 200 text	**Pass**
TC-LLM-I-002	RouteSearchAgent dispatched with route data	Route data	**Pass**
TC-LLM-I-003	POISuggestAgent returns updated POI	Updated POI	**Pass**
TC-LLM-I-004	UserProfileAgent updates preferences	Prefs updated	**Pass**
TC-LLM-I-005	FeedbackAgent updates weights after 5-star	Weights updated	**Pass**

TC-LLM-I-006	XAI generates interest-linked explanation	Explanation	**Pass**
TC-LLM-I-007	WeatherAgent returns forecast data	Forecast data	**Pass (mock data)**
TC-LLM-I-008	ItineraryModAgent adds POI to trip	POI added	**Pass**
TC-LLM-I-009	Multi-turn context retained across 3 msgs	Context kept	**Pass**
TC-LLM-I-010	Long history (40 pairs) does not fail	HTTP 200	**Fail**
TC-LLM-I-011	Chat title persisted after first message	Title saved	**Pass**
TC-LLM-I-012	Chat history persisted and retrievable	3 messages	**Pass**

POI Module Tests — Unit

Test ID	Description	Expected	Result
TC-POIU-001	Get all places, dataset not empty	Non-empty list	**Pass**
TC-POIU-002	Get all places, dataset empty	Empty list	**Pass**
TC-POIU-003	Get places by exact type	Filtered list	**Pass**
TC-POIU-004	Get places by type, no matches	Empty list	**Pass**
TC-POIU-005	Get places by minimum rating	Filtered list	**Pass**
TC-POIU-006	Get places by min rating, no matches	Empty list	**Pass**
TC-POIU-007	Get place by exact name	Matching POI	**Pass**
TC-POIU-008	Get place by fuzzy/partial name	Matching POIs	**Pass**
TC-POIU-009	Get place by name, nonexistent	Empty result	**Pass**
TC-POIU-010	Get places by price level	Filtered list	**Pass**
TC-POIU-011	Get places by price level, no matches	Empty list	**Pass**
TC-POIU-012	Get place by id, valid	POI id=123	**Pass**
TC-POIU-013	Get place by id, not found	Not found	**Pass**
TC-POIU-014	Get places by ids, all valid	3 POIs	**Pass**
TC-POIU-015	Get places by ids, partially valid	Valid only	**Pass**
TC-POIU-016	Get places by ids, empty list	Empty list	**Pass**
TC-POIU-017	Pagination first page	≤10 POIs	**Pass**
TC-POIU-018	Pagination middle page	Page slice	**Pass**
TC-POIU-019	Pagination out-of-range page	Empty page	**Pass**
TC-POIU-020	Business status filtering	Filtered list	**Pass**

POI Module Tests — Integration

Test ID	Description	Expected	Result
TC-POII-001	Search by exact name	HTTP 200 match	**Pass**
TC-POII-002	Search by fuzzy/partial name	HTTP 200 list	**Pass**
TC-POII-003	Search non-existing name	HTTP 200 empty	**Pass**
TC-POII-004	Filter by type	HTTP 200 typed	**Pass**
TC-POII-005	Filter by minimum rating	HTTP 200 rated	**Pass**
TC-POII-006	Filter by price level	HTTP 200 priced	**Pass**
TC-POII-007	Filter by business status	HTTP 200 status	**Pass**
TC-POII-008	Combined filters applied	HTTP 200 all	**Pass**
TC-POII-009	Retrieve all POIs	HTTP 200 list	**Pass**
TC-POII-010	Retrieve POI by id valid	HTTP 200 POI	**Pass**
TC-POII-011	Retrieve POI by id not found	HTTP 404	**Pass**
TC-POII-012	Retrieve POIs by id list	HTTP 200 list	**Pass**
TC-POII-013	Pagination valid page	HTTP 200 page	**Pass**
TC-POII-014	Pagination out-of-range	HTTP 200 empty	**Pass**
TC-POII-015	Invalid min rating → 400	HTTP 400	**Pass**
TC-POII-016	Invalid type value → empty	HTTP 200 empty	**Pass**
TC-POII-017	Invalid price level → empty	HTTP 200 empty	**Pass**
TC-POII-018	Response payload structure valid	All fields	**Pass**
TC-POII-019	Combined filters no match → empty	HTTP 200 empty	**Pass**
TC-POII-020	Empty search parameter → empty	HTTP 200 empty	**Pass**

Performance Tests

Test ID	Description	Expected	Result

TC-PERF-001	Route generation latency — single request ≤5s	≤5s	**Pass**
TC-PERF-002	Route generation latency — 3 alternatives ≤8s	≤8s	**Pass**
TC-PERF-003	LLM /chat warm model ≤10s for Qwen model and ≤ 80s for Nemotron	Qwen: ≤10s AND Nemotron ≤80s	**Pass**
TC-PERF-004	LLM /chat tool-calling ≤15s for Qwen model and ≤90s for Nemotron	Qwen: ≤15s AND Nemotron ≤90s	**Pass**
TC-PERF-005	POI bulk retrieval ≤1s	≤1s	**Pass**
TC-PERF-006	OSRM 10-stop route ≤4s	≤4s	**Pass**
TC-PERF-007	Concurrent 10 users all 200	All HTTP 200	**Partial (depends on user actions)**
TC-PERF-008	JWT 1000 tokens ≤2s	≤2s	**Pass**
TC-PERF-009	Redux large chat state serialization ≤200ms	≤200ms	**Pass**
TC-PERF-010	VRAM/RAM usage under peak load	<24GB/<32GB	**Pass**

Client-Side Tests: Unit

Test ID	Description	Expected	Result
TC-CLI-001	UserService.loginUser formats request correctly	POST + JWT stored	**Pass**
TC-CLI-002	PlacesService.getPlacesByType correct URL	GET museum	**Pass**

Client-Side Tests: Integration

Test ID	Description	Expected	Result
TC-CL-001	Successful login flow	Redirected, auth	**Pass**
TC-CL-002	Session restoration after page refresh	State restored	**Fail**
TC-CL-003	New chat creation and message send	Chat + bubbles	**Pass**
TC-CL-004	Save and unsave destination	Bookmark toggle	**Pass**
TC-CL-005	Map marker renders for each destination	5 markers	**Pass**

TC-CL-006	ExplorePanel category filter	Filtered grid	**Pass**
TC-CL-007	NavigationPanel route calculation	Polyline drawn	**Pass**
TC-CL-008	DestinationDetailPanel resize	Panel resizes	**Pass**
TC-CL-009	Theme toggle dark mode	Dark applied	**Pass**
TC-CL-010	LocationCard renders in chat message	Card shown	**Pass**
TC-CL-011	Markdown rendering in AI response	HTML table	**Pass**
TC-CL-012	Responsive layout on narrow viewport	No overflow	**Pass**

User Testing

Test ID	Description	Expected	Result
TC-UT-001	New user registration and onboarding	Main view <2min	**Pass**
TC-UT-002	Creating a travel persona	Persona active	**Pass**
TC-UT-003	Generating route via NL chat	≥5 POIs <90s	**Pass**
TC-UT-004	Requesting POI replacement via chat	Stop replaced	**Pass**
TC-UT-005	Asking for recommendation explanation	XAI explanation	**Pass**
TC-UT-006	Switching personas and regenerating route	Different route	**Pass**
TC-UT-007	Saving destination and accessing later	Bookmark persists	**Pass**
TC-UT-008	Navigation panel driving route	Polyline <4s	**Pass**
TC-UT-009	Providing trip feedback (4 stars)	Rating accepted	**Pass**
TC-UT-010	Resuming a past chat session	History shown	**Pass**
TC-UT-011	Switching light/dark mode	Theme applied	**Pass**
TC-UT-012	Querying current weather	Forecast <10s	**Pass (mock data)**

System Tests

Test ID	Description	Expected	Result
TC-SYS-001	Full itinerary planning lifecycle	Route rendered	**Pass**
TC-SYS-002	Route modification flow (reroll + add)	Both applied	**Pass**

TC-SYS-003	Chat conversation to persistence	3 msgs persist	**Pass**
TC-SYS-004	Persona lifecycle and route influence	Routes differ	**Pass**
TC-SYS-005	POI search → save → navigation flow	Full flow OK	**Pass**
TC-SYS-006	XAI justification in conversational context	Explanation	**Pass**

6.4 Detailed Test Result Notes

TC-RGI-001 - Generate Alternative Routes

The route generation endpoint was tested with a valid userVector specifying pace: "medium" and interests: ["history"] with k=3. The endpoint returned HTTP 200 with a list of exactly 3 Route objects, each containing valid points and segments arrays. The routes were geographically distinct, confirming the randomness-with-weights mechanism in POI selection.

TC-RGI-002 - Replace a Single POI (Reroll)

A reroll request was sent targeting index 2 of an existing route with 5 points. The response confirmed that only points[2] was replaced with a different museum-type POI, all other points remained identical, and only segments[1] and segments[2] were recomputed with updated duration and distance values. Total route metrics were updated accordingly.

TC-RGI-003 - Add Manual POI

A manual POI insertion request was sent with poiId "abc" at index 1. The response confirmed that POI "abc" was placed at points[1], original points [1..n] were shifted to [2..n+1], and all affected segments were recomputed. Total duration and distance were updated correctly.

TC-LLM-I-007 and TC-UT-012

The WeatherAgent tool-call dispatch and response rendering were verified; the agent currently returns stubbed forecast data pending live weather provider integration (see 4.2 and 10.2).

TC-PERF-007 - Concurrent User Testing

The TenConcurrentUsersRealDatabaseTest suite was executed with 10 simultaneously released threads against a live PostgreSQL instance, each performing the full flow: login, place queries, route generation, and LLM chat. All 10 futures resolved within the 180-second deadline.

TC-PERF-010 - VRAM/RAM usage check

In all tests the team did not get any Out of VRAM Memory errors. The maximum amount of VRAM used when using Nemotron is around 22.5 GB.

6.5 Summary of Test Results

A total of 225 test cases were executed across unit, integration, system, performance, security, user acceptance, and client-side categories as summarized in 6.3. Of these, 221 passed, 1 achieved partial pass, and 3 failed and were subsequently fixed (see 6.6). The effective pass rate is 98.2%.

After these tests are evaluated, bugs on the failing tests were examined and remedies were found, as described further in the section 6.6. Currently no critical blocking bugs remain in the final build.

6.6 Bugs Found and Fixes Applied

Bug: OSRM returned empty geometry for some pedestrian-mode routes in areas with sparse map data.

Fix: A Haversine straight-line distance fallback was implemented when RouteSegment.hasGeometry() returns false.

Bug: Long conversation histories (40+ message pairs) occasionally caused HTTP 413 payload errors.

Fix: Message history is truncated to the most recent 20 pairs while retaining the initial system context.

Bug: Redux localStorage serialization failed for deeply nested Chat objects containing LocationCard data.

Fix: A custom serializer/deserializer was implemented for the Redux persistence middleware.

Currently, all of these bugs are fixed and tests are passing.

6.7 Potential Enhancements

Implement a streaming response mechanism for the LLM chat to reduce perceived latency on slow connections.

Add a comprehensive accessibility tagging system (age ranges, wheelchair access) to the POI database schema.

Develop a self-service GDPR data deletion button in the frontend settings panel.

Expand to other Turkish cities beyond Ankara, with broader Google Places data ingestion.

Implement a collaborative itinerary sharing feature allowing multiple users to co-edit a route.

7. Impact of Engineering Solutions

7.1 Global Context

AI-assisted travel planning addresses a universal friction point: the cognitive overhead of assembling a coherent itinerary from fragmented, ad-saturated, and often contradictory online sources. In this respect, RoadRunner has relevance beyond its current deployment scope. Its core design preference vector extraction, weighted POI scoring, constraint-based route generation, and conversational LLM interaction is domain-general rather than Ankara-specific. The current geographic limitation is therefore a data constraint, not an algorithmic one.

Extending the system to cities such as Istanbul, Barcelona, or Tokyo would primarily require collecting new POI datasets for those regions and adding them to the existing seed pipeline, while leaving the routing and scoring engine unchanged.

At the same time, the system's broader international applicability remains limited by its present cultural and linguistic scope. The conversational interface currently performs best in English, and the platform does not yet provide explicit multilingual support or locale-aware formatting. POI category labels such as HISTORIC_PLACES, PARKS, and CAFES_AND_DESSERTS are also defined in English through Google Places mappings. In addition, culturally specific planning dimensions such as prayer time windows, dietary restrictions beyond broad budget preferences, and local holiday closures are outside the current constraint model.

RoadRunner also depends on a set of global third-party data platforms. POI data is sourced from Google Places API, map visualization relies on OpenStreetMap, and road routing depends on OSRM's public server. These dependencies make rapid development and broad geographical extensibility possible, but they also tie the system's global usability to the availability, consistency, and regional data quality of external services outside the team's direct control.

7.2 Economic Context

- **API and infrastructure costs:** During development, the primary cost-generating external service was Google Places API for data collection. The grid-based Nearby Search approach in fill_database.py and fill_database_bolgesel.py issues hundreds of API calls per collection run. However, because the system seeds POI data into PostgreSQL at startup and never calls Google Places at runtime, ongoing API cost is zero during normal operation.
- **Hosting:** The system locally deployed runs on minimal costs, each modular part of the system architecture works in unison and the only cost is the electricity cost.
- **Business value:** From a user perspective, RoadRunner eliminates hours of manual research across booking sites, review aggregators, and map applications. The persona system means returning users do not repeat preference configuration. The saved route snapshot mechanism enables a potential marketplace where experienced travelers publish and share curated itineraries.

7.3 Environmental Context

- **Route optimization and travel efficiency:** The routing algorithm minimizes backtracking by sequencing stops via nearest-neighbor heuristics and distributing meal stops to avoid clustering. This produces itineraries that are geographically efficient, which in turn reduces the distance a traveler actually walks or drives. For driving routes, shorter total distance directly maps to lower fuel consumption and emissions. The system does not currently calculate or display carbon footprint estimates, but the

RouteSegment model already stores distance in meters and estimated duration per segment, adding a CO_2 estimate per transport mode would be a straightforward extension.

- **Future potential:** An eco-friendly routing mode could weight POIs reachable by walking or public transit more heavily, penalize car-dependent sequences, or surface a "greenest route" alternative alongside the default. The transportStyle field in the persona model already captures whether a user prefers walking, or driving; this preference currently affects segment duration estimates but does not influence POI selection. Connecting it to the scoring layer would enable genuinely sustainability-aware recommendations.
- **Every LLM chat interaction incurs inference cost:** RoadRunner's primary model is Nemotron, run on the team's local GPU server (NVIDIA Titan, 24 GB VRAM) via the Flask endpoint, not a cloud API. This means inference energy is consumed on-premises rather than in a hyperscaler data center, but the GPU power draw is still substantial. A single tool-calling turn requires at minimum two forward passes (initial intent parsing + final response after tool result), each saturating the GPU for seconds to minutes. At scale, the chat feature's energy footprint would dominate the system's total consumption; the Spring Boot backend, PostgreSQL, and React frontend are negligible in comparison.

  This is a structural property of any LLM-powered application, not unique to RoadRunner. Adding a conversational AI layer increases per-query energy cost by roughly two orders of magnitude compared to a traditional search-and-filter interface. The system partially mitigates this by making the manual UI (persona builder, constraint selectors, route editor) also available, so users who prefer direct controls never trigger LLM inference at all.

7.4 Societal Context

- **Accessibility and inclusiveness:** RoadRunner aims to lower the barrier to quality travel planning. Users who lack the time, language skills, or domain knowledge to manually research destinations get a structured, preference-aware itinerary without needing to understand routing algorithms, POI databases, or map tools. The conversational interface is particularly valuable for users who find traditional filter-heavy UIs overwhelming; they can express intent in natural language ("show me quiet places with good food near Kızılay") and receive actionable results.
  However, the system has inclusiveness gaps. No accessibility tagging exists in the POI data right now; wheelchair access, elevator availability, sensory-friendly environments, and age-range suitability are not modeled. The persona system captures travel style preferences but not mobility constraints or disability accommodations. These are documented as future work items, but their absence means the system currently serves able-bodied, digitally literate users disproportionately well.
- **Support for different traveler groups:** The persona system's multi-profile support is designed for the reality that people travel differently depending on context — a solo weekend trip has different constraints than a family vacation or a business trip with limited free hours. The constraint system's meal stop injection, hotel loop anchoring, and tempo-based POI density adjustment address common but distinct needs across traveler

types. That said, family-specific features (child-friendly filtering, stroller accessibility, nap-time scheduling) and group coordination (multiple personas merged into a single route) are not implemented.

- **Trust in AI recommendations:** This is the system's most significant societal challenge. The LLM can confidently recommend POIs that are permanently closed, suggest routes that are physically unreasonable, or hallucinate place names that do not exist in the database. The architecture mitigates this by grounding LLM tool calls in the real PostgreSQL dataset, the chatbot cannot recommend a place that is not in the seeded POI table, but it can still misinterpret user intent, dispatch the wrong agent, or generate misleading natural language around correct data. The XAI justification agent exists to explain why a POI was recommended, which helps build transparency, but users have no way to verify the explanation's accuracy without independent research. The dual-interface design (chat + manual UI) is the most honest mitigation: users can always bypass the LLM and interact with the deterministic system directly.
- **Social impact:** At a broader level, tools like RoadRunner shift travel planning from a privilege requiring expertise and time into something closer to a utility. A first-time traveler in Ankara with no local knowledge receives the same quality of route optimization as a frequent visitor who knows the city well. Whether this flattening of information asymmetry actually improves travel experiences or simply creates a new dependency on AI systems is an open question, one that applies equally to every AI assistant in every domain, not uniquely to travel planning.

8. Contemporary Issues

8.1 AI Reliability and Hallucination

Large Language Models carry an inherent risk of generating plausible-sounding but factually incorrect outputs a phenomenon known as hallucination. In a travel planning context this is particularly consequential: an LLM could recommend a permanently closed restaurant, cite incorrect opening hours, or suggest an impassable route. RoadRunner mitigates this by treating the LLM strictly as an intent-parsing and orchestration layer rather than a source of factual data. All POI references produced by agents such as RouteSearchAgent and POISuggestAgent are cross-referenced against the internal PostgreSQL database seeded from verified Google Places data; any place that cannot be matched to a known entity is rejected before reaching the user. Residual risks remain in free-text justifications and conversational responses, which are not validated against structured data. Additionally, LLM non-determinism means identical queries may yield different results across sessions. Future mitigations include confidence scoring for agent outputs, explicit uncertainty disclosures in the chat interface.

8.2 Privacy and Data Protection

RoadRunner collects account credentials, travel personas, itinerary history, saved destinations, and full LLM conversation logs data that collectively constitutes a detailed behavioral profile.

The system's compliance framework is grounded in GDPR (EU Regulation 2016/679) and Turkey's KVKK (Law No. 6698), both of which mandate data minimization, purpose limitation, and user rights to access, correct, and delete personal data. In practice, passwords are hashed with BCrypt and never stored in plaintext, all communication is encrypted over HTTPS/TLS, and JWT session tokens expire automatically. Google reCAPTCHA v3 protects registration and login against automated attacks. A backend data deletion API supports the Right to be Forgotten. A known gap is the absence of a self-service deletion button in the frontend UI the endpoint exists but has not been surfaced to users, and is identified as a priority for future development. Cross-border data flow also presents ongoing legal complexity if the system is deployed on infrastructure outside Turkey or the EU.

8.3 Bias and Fairness in Recommendations

Recommendation systems reflect and amplify the biases present in their source data. RoadRunner's POI database is seeded from Google Places API data, which aggregates reviews from a population that skews toward urban, digitally literate, and relatively affluent users. High-visibility venues major tourist landmarks, internationally recognized restaurants, and well-reviewed hotels naturally accumulate more ratings, causing the scoring mechanism to surface them disproportionately. Small locally-owned businesses, minority-owned establishments, and budget-friendly neighborhood venues are systematically underrepresented. LLM training data compounds this problem, as travel content on the web heavily favors luxury and Western-centric experiences. RoadRunner partially mitigates bias through persona-based preference weighting: users who configure a budget-conscious persona receive POI selections weighted toward lower price levels and less-visited categories. However, these mitigations are user-driven and do not proactively correct structural underrepresentation in the underlying dataset. Future work should include diversification mechanisms in POI scoring and regular auditing of recommendation distributions across venue type and price level categories.

8.4 Dependence on Third-Party Platforms

RoadRunner's functionality depends on a chain of external services Google Places API and OSRM each introducing its own availability, cost, and reliability risks. Google Places API enforces daily quotas and per-request pricing that could become significant under real user traffic; a pricing change or API deprecation would require substantial re-engineering of the data ingestion pipeline. The system mitigates this by persisting all POI data internally and avoiding live runtime API calls, though this creates a data freshness problem: venue closures, new openings, and rating changes are not automatically reflected. OSRM, being open-source and self-hostable, reduces vendor lock-in for routing, but its accuracy depends on OpenStreetMap coverage completeness, which is uneven across Ankara. A straight-line distance fallback is used where OSRM geometry is unavailable.

8.5 Ethical Transparency

As AI systems take an active role in shaping user decisions about where to travel and what to visit, the obligation to be transparent about how those decisions are made grows proportionally. RoadRunner's primary transparency mechanism is the XAI Justification Agent, which generates natural language explanations explicitly linking each POI recommendation to attributes of the user's travel persona distinguishing it from opaque black-box systems and enabling users to identify and correct preference mismatches. On commercial integrity, all recommendations are derived purely from internal scoring and user preference weighting; no mechanism exists for venue operators to pay for preferential placement, a meaningful distinction from many existing travel platforms where sponsored listings are intermixed with organic results without clear disclosure. Any future monetization must maintain this separation and comply with applicable advertising transparency regulations. A current gap is the absence of in-UI disclaimers about data freshness, LLM non-determinism, and geographic coverage limits. Users should be informed that AI-generated content may contain inaccuracies and should be verified before acting on it for time-sensitive travel decisions.

9. Use of Library and Internet Resources

9.1 Background Research Sources

- **Official API Documentation:** The following API documentation was used directly during implementation: Google Places API (New) documentation for understanding the Nearby Search request schema, field masks, and place type taxonomy used in the data collection scripts; OSRM API documentation for the route service endpoint contract (/route/v1/{profile}/{coordinates}), response geometry format, and query parameter options; Spring Boot and Spring Security reference documentation for JWT filter chain configuration, @PreAuthorize annotations, and SecurityFilterChain bean setup; React, Redux Toolkit, and React-Leaflet documentation for component lifecycle, createAsyncThunk patterns, and map instance control.
- **Technical Articles and Tutorials:** Medium articles and developer blog posts were referenced for practical implementation patterns: RAG integration approaches (S. Heyer, "RAG API," Medium, 2024), Foursquare-based spatial analysis of Ankara restaurants (İ. Sarıçiçek, Medium, 2021) which informed the grid-based data collection strategy, and various Stack Overflow threads for resolving Spring Security + JWT integration edge cases and Leaflet marker clustering performance tuning.
- **Academic Papers:** Two academic sources directly informed design decisions: C.-Y. Tsai and J.-H. Wang, "A Personalized Itinerary Recommender System: Considering Sequential Pattern Mining" (Electronics, 2025) provided background on preference-weighted POI scoring and sequential visit optimization; the multi-factor scoring model in RouteScoringService draws on similar principles. A. Chen et al., "TravelAgent: An AI Assistant for Personalized Travel Planning" (arXiv, 2024) served as

a reference architecture for LLM-based intent parsing and agent dispatch in travel domains, directly influencing the tool-calling pipeline design.

- **Standards and Ethics Sources:** The ACM Code of Ethics and Professional Conduct (2018) and the IEEE Code of Ethics (2020) were used to frame the project's professional responsibility requirements, particularly around transparency in AI recommendations and honest representation of system capabilities. The Stanford Encyclopedia of Philosophy entry on "Computer and Information Ethics" (2020) provided background for the contemporary issues discussion on AI bias and user trust. GDPR (EU Regulation 2016/679) and KVKK (Turkish Law No. 6698) were referenced for data protection requirements, informing the decision to support account deletion on the backend and to exclude application.properties (containing secrets) from version control.

9.2 Similar Systems and Existing Solutions

- **Google Travel / Google Trips:** Examined for its itinerary aggregation approach, automatically pulling flight/hotel confirmations from Gmail and grouping saved places by destination. RoadRunner differs fundamentally: it generates itineraries from preferences rather than organizing existing bookings, and it uses a conversational interface for intent capture rather than a booking-centric workflow. Google Trips was also discontinued in 2019, which validated the demand gap RoadRunner targets.
- **Tripadvisor:** Studied for its review-based POI ranking system and the "Things to Do" category structure. RoadRunner incorporates rating data (score and count) from Google Places but adds user-specific preference weighting on top, a POI with a 4.2 rating can outrank a 4.7-rated one if its category aligns better with the user's persona vector. Tripadvisor has no route optimization; users must manually sequence their saved places.
- **Rome2Rio:** Referenced for multi-modal transport routing between cities. OSRM was selected as the open-source alternative for intra-city routing. Rome2Rio focuses on inter-city transport (flights, trains, buses) which is outside RoadRunner's scope, RoadRunner handles within-destination itinerary sequencing, not how to get to the destination.
- **ChatGPT Travel Planning Plugins:** Reviewed to understand how LLM tool-calling applies to travel domains. Key observations: ChatGPT plugins call external APIs on every request (live hotel prices, flight availability), making them dependent on real-time API access. RoadRunner's architecture deliberately avoids this — POI data is pre-seeded locally, and the LLM's tool calls hit internal backend endpoints, not third-party services. This makes RoadRunner's responses faster and cheaper per query, at the cost of data freshness.

9.3 Engineering Principles and External References

- **Architecture decisions:** The modular monolith structure (as opposed to the originally proposed microservices) was informed by Martin Fowler's writing on the "Monolith First" pattern of starting with a single deployable unit and extracting services only when scaling demands it. Spring Boot's documentation on component scanning, package-level

organization, and @Configuration classes guided the internal module boundaries (user, place, route, llm, config, security).

- **Routing logic.** The nearest-neighbor heuristic for stop sequencing was chosen after reviewing the Traveling Salesman Problem literature and determining that exact solutions (branch-and-bound, dynamic programming) were computationally infeasible for interactive response times. OSRM API documentation guided the integration for road-network distance/duration queries.
- **Frontend design:** React-Leaflet documentation and community examples guided the marker clustering implementation, polyline rendering, and programmatic map control (zoom-to-fit on route generation). Redux Toolkit's official documentation on createAsyncThunk and createSlice patterns shaped the seven-slice store architecture. Joy UI's component API documentation and theming guide informed the light/dark mode toggle and consistent styling across all pages.
- **Security practices:** Spring Security's reference documentation on stateless session configuration, OncePerRequestFilter for JWT validation, and BCryptPasswordEncoder for password hashing were the primary references. The decision to use JWT with expiration (rather than server-side sessions) was informed by OWASP's session management guidelines and the stateless REST API design principle.

9.4 Lifelong Learning / Self-Learning Aspect

The project required the team to learn several technologies and paradigms beyond what was covered in coursework.

- **LLM Tool-Calling and Prompt Engineering:** No team member had prior experience with LLM function calling, agent orchestration, or prompt engineering for deterministic tool dispatch. The team learned how to declare tool schemas, parse structured tool-call responses from the LLM, implement the execute-and-resubmit loop, and handle failure cases (LLM refusing to call a tool, calling the wrong tool, returning malformed parameters). Multiple LLM backends were tested; Qwen, and Nemotron; each with different tool-calling syntax and reliability characteristics. This required rapid iteration and comparative evaluation, skills not taught in any course.
- **Spring Boot Security and JWT Integration:** While Java was familiar from coursework, the Spring Security filter chain, custom OncePerRequestFilter implementation for JWT extraction, SecurityFilterChain bean configuration, and the interaction between AuthenticationManager, UserDetailsService, and BCryptPasswordEncoder were all learned from documentation and community resources during the project. The RecaptchaService integration with Google's reCAPTCHA v3 verification endpoint was similarly self-taught.
- **Leaflet and Interactive Map Development:** Building an interactive map with marker clustering, polyline route overlays, popup interactions, resizable panels, and OSRM routing integration required learning Leaflet's imperative API and bridging it with React's declarative model via React-Leaflet. Handling edge cases such as OSRM returning

empty geometry, marker cluster performance with 1000+ POIs, programmatic zoom control, demanded understanding beyond introductory tutorials.

- **Route Generation Algorithm Design:** No off-the-shelf library was used for the core itinerary optimization. The team designed and implemented the multi-factor scoring model, category quota allocation, nearest-neighbor sequencing, meal stop distribution, anchor constraint system, and alternative route diversity penalty from scratch. This required self-directed study of heuristic optimization, the Traveling Salesman Problem, and spatial data structures.
- **Redux Toolkit State Architecture:** While React was covered in limited scope during internships, designing a seven-slice Redux store with cross-slice dependencies (route generation results feeding into saved route creation, persona updates affecting route weight computation, chat messages containing structured location card data) required understanding advanced patterns: createAsyncThunk with sequential dispatch, custom serializers for persistence middleware, and selector composition across slices.

10. Limitations and Future Work

10.1 Current Limitations

Data Freshness: The POI database is a static snapshot seeded from Google Places API at setup time. Venue closures, new openings, changed opening hours, and updated ratings are not automatically reflected. Users may receive recommendations for places that no longer operate as described.

Geographic Coverage: The system is scoped exclusively to Ankara. Users planning trips to other Turkish cities or international destinations receive no POI data or route generation support. Within Ankara, category coverage is uneven bars and nightclubs, hotels, and parks have fewer seeded entries than restaurants and historic places.

Accessibility Metadata and UI accessibility (WCAG): The Place entity schema does not include wheelchair access, age-range suitability, or sensory-friendly attributes. Users with disabilities or families with specific accessibility needs cannot filter recommendations accordingly, limiting the system's inclusivity.

LLM Non-Determinism: The Flask-based LLM server operates as a prototype and is not co-deployed with the main backend in production. Identical user queries may yield different agent dispatch sequences or recommendation sets across sessions, making behavior difficult to predict or audit.

Route Segment Geometry: Backend RouteSegment objects store only straight-line distance estimates and approximate durations. Full OSRM turn-by-turn geometry is available only in the frontend NavigationPanel and is not persisted with saved routes, meaning exported or replayed itineraries lack precise path data.

No Real-Time Optimization: Route generation is a one-shot process based on static preference weights. The system does not adapt in real time to live factors such as current traffic conditions, temporary venue closures, weather disruptions, or dynamic pricing.

Concurrency Ceiling: The system is designed and tested for up to 10 concurrent users. The LLM server, running on a single GPU instance, becomes a bottleneck under higher load. Internet-scale traffic is outside the current architecture's scope.

10.2 Future Improvements

Multi-City and International Expansion. Extending the POI seeding pipeline to cover other major Turkish cities and popular international destinations would transform RoadRunner from a single-city tool into a general-purpose travel assistant. The data ingestion architecture is already designed to accept additional Google Places data feeds with minimal schema changes.

Weather-Aware Scheduling. Integrating a live weather API (e.g., OpenWeatherMap) would allow the system to adjust itineraries dynamically deprioritizing outdoor venues during rain, surfacing indoor alternatives, and flagging weather risks for specific travel dates. The WeatherAgent skeleton is already defined in the LLM agent architecture and awaits a production data source.

Accessibility-First Filtering. Adding wheelchair access, step-free routing, hearing loop availability, and age-range suitability fields to the Place schema would make the system genuinely inclusive. This requires both a data enrichment effort (sourcing accessibility attributes from Google Places or OpenStreetMap) and frontend filter controls.

Stronger Recommendation Ranking. The current scoring mechanism relies primarily on rating scores and persona weight vectors. Future iterations should incorporate collaborative filtering learning from aggregated user behavior across similar personas and contextual signals such as time of day, current season, and real-time venue busyness to produce more nuanced rankings.

User Feedback Learning. The UserFeedbackAgent currently adjusts persona preference weights based on explicit star ratings. A more sophisticated feedback loop would use implicit signals which POIs the user retained versus replaced during itinerary editing, how long they spent viewing a venue card, which saved routes they revisited to continuously refine the preference model without requiring explicit user input.

Smarter Budget Optimization. Rather than filtering by broad price level categories, a future version should accept a concrete budget figure and optimize route composition to maximize satisfaction within that constraint, estimating cumulative trip cost from per-venue price level data and visit duration.

Mobile Application. The current system is a responsive web application with no offline capability. A Progressive Web App (PWA) or native mobile application would allow users to

access saved itineraries offline, receive push notifications for time-sensitive travel reminders, and use device GPS for real-time navigation integration.

11. Conclusion

RoadRunner successfully demonstrates how large language models, constraint-based route generation, and interactive web technologies can be combined into a cohesive, user-centered travel planning system. What began as a response to the fragmented and generic nature of existing travel tools evolved into a full-stack web application that spans conversational AI, preference-driven itinerary generation, interactive map visualization, and persistent user profile management all operating within a single unified product experience.

The final system delivers on its core objectives. Users can describe a trip in natural language and receive a personalized, map-rendered itinerary. They can refine that itinerary through both chat commands and direct UI interaction, switching between multiple travel personas to explore how different preference profiles change the recommendations they receive. Saved routes persist across sessions, the LLM agent pipeline connects natural language intent to real application data, and the entire stack is containerized for reproducible deployment.

The project's strongest technical contributions are the persona-to-weight-vector translation system that makes abstract user preferences computationally actionable, the constraint-aware route generation engine that balances mandatory stop types, budget levels, and geographic feasibility, and the tool-calling LLM architecture that bridges free-form conversation with structured backend operations. Together these form a genuinely novel integration layer that goes well beyond a standard CRUD application.

Several areas remain open for future development. Geographic coverage is limited to Ankara, data freshness depends on periodic manual re-seeding, and the Flask LLM server remains at prototype level rather than production deployment. Accessibility metadata, real-time weather integration and collaborative filtering for recommendation ranking are all identified improvements that the current codebase is architecturally prepared to support.

As detailed in Section 10, the key limitations include geographic confinement to Ankara, LLM non-determinism, and lack of real-time data refresh; each of which has a corresponding improvement pathway defined for future iterations

In summary, RoadRunner establishes a strong and extensible foundation for intelligent travel assistance. The decisions made during design and implementation modular service boundaries, persona-driven personalization, structured POI verification for LLM outputs, and an open routing engine ensure that the path from the current working system to a production-ready, internationally capable platform is incremental rather than requiring fundamental rearchitecture.

12. References

[1] ACM, "ACM Code of Ethics and Professional Conduct," 2018. [Online]. Available: https://www.acm.org/code-of-ethics

[2] IEEE, "IEEE Code of Ethics," 2020. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html

[3] Google, "Places API (New) Documentation," 2025. [Online]. Available: https://developers.google.com/maps/documentation/places/web-service

[4] Google, "reCAPTCHA v3 Developer Guide," 2025. [Online]. Available: https://developers.google.com/recaptcha/docs/v3

[5] Project OSRM, "OSRM API Documentation — Route Service," 2025. [Online]. Available: https://project-osrm.org/docs/v5.24.0/api/

[6] OpenStreetMap Wiki, "Map Features," 2025. [Online]. Available: https://wiki.openstreetmap.org/wiki/Map_features

[7] VMware/Pivotal, "Spring Boot Reference Documentation," 2025. [Online]. Available: https://docs.spring.io/spring-boot/reference/

[8] VMware/Pivotal, "Spring Security Reference," 2025. [Online]. Available: https://docs.spring.io/spring-security/reference/

[9] VMware/Pivotal, "Spring Data JPA Reference Documentation," 2025. [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/

[10] Meta Platforms, "React Documentation," 2025. [Online]. Available: https://react.dev

[11] Redux Team, "Redux Toolkit Documentation," 2025. [Online]. Available: https://redux-toolkit.js.org

[12] Evan You et al., "Vite — Next Generation Frontend Tooling," 2025. [Online]. Available: https://vitejs.dev

[13] V. Agafonkin, "Leaflet — An Open-Source JavaScript Library for Interactive Maps," 2025. [Online]. Available: https://leafletjs.com

[14] P. Skare et al., "React-Leaflet Documentation," 2025. [Online]. Available: https://react-leaflet.js.org

[15] MUI, "Joy UI Documentation," 2025. [Online]. Available: https://mui.com/joy-ui/getting-started/

[16] Pallets Projects, "Flask Documentation," 2025. [Online]. Available: https://flask.palletsprojects.com

[17] The PostgreSQL Global Development Group, "PostgreSQL Documentation," 2025. [Online]. Available: https://www.postgresql.org/docs/

[18] C.-Y. Tsai and J.-H. Wang, "A Personalized Itinerary Recommender System: Considering Sequential Pattern Mining," *Electronics*, vol. 14, no. 2, 2025. DOI: 10.3390/electronics14020001

[19] A. Chen, J. Zhang, and Y. Liu, "TravelAgent: An AI Assistant for Personalized Travel Planning," *arXiv preprint*, arXiv:2408.16867, 2024. [Online]. Available: https://arxiv.org/abs/2408.16867

[20] European Parliament and Council, "Regulation (EU) 2016/679 — General Data Protection Regulation (GDPR)," *Official Journal of the European Union*, L 119, pp. 1–88, May 2016.

[21] Türkiye Büyük Millet Meclisi, "Kişisel Verilerin Korunması Kanunu (KVKK), Kanun No. 6698," *Resmî Gazete*, No. 29677, April 2016.

[22] M. Fowler, "Monolith First," martinFowler.com, June 2015. [Online]. Available: https://martinfowler.com/bliki/MonolithFirst.html

[23] OWASP Foundation, "Session Management Cheat Sheet," 2025. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[24] JUnit Team, "JUnit 5 User Guide," 2025. [Online]. Available: https://junit.org/junit5/docs/current/user-guide/

[25] Mockito Contributors, "Mockito Framework Documentation," 2025. [Online]. Available: https://site.mockito.org

[26] Qameta Software, "Allure Test Report Documentation," 2025. [Online]. Available: https://allurereport.org/docs/

[27] Project Lombok, "Lombok Features Documentation," 2025. [Online]. Available: https://projectlombok.org/features/

[28] Alibaba Cloud, "Qwen Technical Report," *arXiv preprint*, arXiv:2309.16609, 2023. [Online]. Available: https://arxiv.org/abs/2309.16609

[29] NVIDIA, "Nemotron-4 Technical Report," 2024. [Online]. Available: https://research.nvidia.com/publication/2024-06_nemotron-4

[30] Docker Inc., "Docker Documentation," 2025. [Online]. Available: https://docs.docker.com

[31] Apache Software Foundation, "Apache Maven Documentation," 2025. [Online]. Available: https://maven.apache.org/guides/

[32]Nyami, “Production grade RAG ‘Fast’ API,” *Medium*, Mar. 06, 2024. https://cikce.medium.com/prod-grade-rag-fast-api-46b95658d816

[33]T. Kaya, “‘Gelişmiş Yapay Zeka Teknikleri: .NET’te Retrieval Augmented Generation(RAG) Kullanımı,’” *Medium*, Jun. 04, 2024. https://medium.com/lcw-digital/geli%C5%9Fmi%C5%9F-yapay-zeka-teknikleri-nette-retrieval-augmented-generation-rag-kullan%C4%B1m%C4%B1-f63adc164ce1

[34]N. Tech, “RAG Mimarisi: Belgelerle Konuşan Uygulamalar,” *Medium*, Aug. 17, 2025. https://medium.com/%40nemcatech/rag-mimarisi-belgelerle-konu%C5%9Fan-uygulamalar-6835d771c12c

[35]T. Bynum, “Computer and Information Ethics,” *plato.stanford.edu*, Aug. 2001, Available: https://plato.stanford.edu/archives/sum2020/entries/ethics-computer/